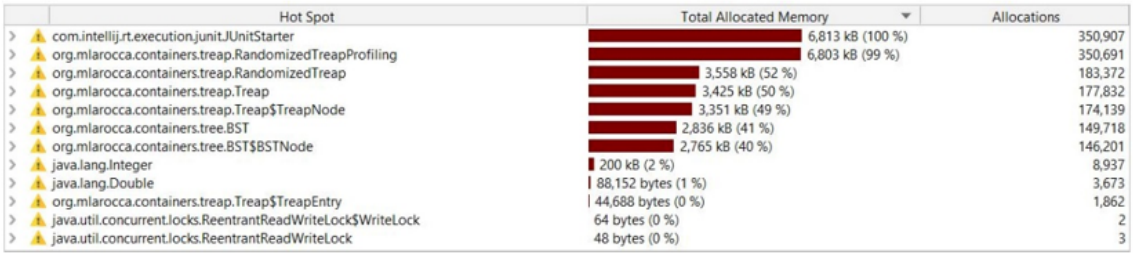


Figure 3.23 Cumulative memory allocation for `BST` and `RandomizedTreap`, in the most generic case, with random insertions and removals.

Once we have established that, we can therefore be happy by profiling memory allocation for the most generic case, where the two trees are both approximately balanced. Figure 3.23 shows the cumulative memory allocated for the whole test for instances of the two classes: we can see that `RandomizedTreap` requires slightly more than twice the memory of `BST`; this is obviously not ideal, but quite expected, considering that each node of a treap will hold a key (an `Integer`, in this test), plus a `Double` for the priority.

If we try a different kind of data for the keys, for instance `String`, we can see that the difference become much smaller, as shown in figure 3.24: just a ratio of 1.25 when storing strings between 4 and 9 characters as keys.



Hot Spot	Total Allocated Memory	Allocations
com.intellij.rt.execution.junit.JUnitStarter	6,813 kB (100 %)	350,907
org.mlarocca.containers.treap.RandomizedTreapProfiling	6,803 kB (99 %)	350,691
org.mlarocca.containers.treap.RandomizedTreap	3,558 kB (52 %)	183,372
org.mlarocca.containers.treap.Treap	3,425 kB (50 %)	177,832
org.mlarocca.containers.treap.Treap$TreapNode	3,351 kB (49 %)	174,139
org.mlarocca.containers.tree.BST	2,836 kB (41 %)	149,718
org.mlarocca.containers.tree.BST$BSTNode	2,765 kB (40 %)	146,201
java.lang.Integer	200 kB (2 %)	8,937
java.lang.Double	88,152 bytes (1 %)	3,673
org.mlarocca.containers.treap.Treap$TreapEntry	44,688 bytes (0 %)	1,862
java.util.concurrent.locks.ReentrantReadWriteLock$WriteLock	64 bytes (0 %)	2
java.util.concurrent.locks.ReentrantReadWriteLock	48 bytes (0 %)	3

Figure 3.24 Cumulative memory allocation for `BST` and `RandomizedTreap` when storing strings between 4 and 10 characters.